



Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Курсова робота

з дисципліни «Інженерія програмного забезпечення»

на тему:

**«Система діалогової взаємодії з великими мовними
моделями з веб- та консольним інтерфейсами»**

Виконав: студент 2 курсу ФІОТ, гр. ІО-41

Давидчук А. М.

Залікова книжка № 4106

Укладач: старший викладач *Васильєва М. Д.*

ЗАВДАННЯ НА КУРСОВУ РОБОТУ

Давидчуку Артему Михайловичу

1. **Тема роботи:** «Система діалогової взаємодії з великими мовними моделями з веб-та консольним інтерфейсами».
2. **Термін здачі закінченої роботи:** 13 травня 2026 р.
3. **Завдання курсової роботи:** розробити програмну систему для ведення діалогів з локальною великою мовною моделлю, сформулювати функціональні та нефункціональні вимоги, спроектувати архітектуру програмного продукту, реалізувати web-та CLI-інтерфейси, забезпечити збереження історії діалогів у PostgreSQL, реалізувати інтеграцію з локальною LLM, streaming-генерацію, runtime-керування моделлю, підтримку generation presets і LoRA/QLoRA-адаптерів, а також розробити модульні та інтеграційні тести для ключових компонентів системи.
4. **Зміст розрахунково-пояснювальної записки:** анотація, вступ, постановка задачі, аналітичний огляд предметної області, проектування програмного додатку, розробка програмного додатку, тестування програмного додатку, висновки, список використаних джерел.
5. **Перелік графічного матеріалу:** діаграма архітектури системи, ER-діаграма бази даних, UML-діаграма прецедентів, UML-діаграма класів, UML-діаграма послідовності надсилення повідомлення.
6. **Дата видачі завдання:** лютий 2026 р.

Календарний план виконання курсової роботи

№	Найменування етапу	Термін виконання	Примітка
1	Отримання теми та завдання	лютий 2026	виконано
2	Підбір та вивчення літератури	лютий 2026	виконано
3	Формування технічного завдання	березень 2026	виконано
4	Формування вимог до програмної системи	березень 2026	виконано
5	Розробка UML-діаграм і архітектури системи	березень 2026	виконано
6	Реалізація програмного додатку	квітень 2026	виконано
7	Тестування програмного додатку	травень 2026	виконано
8	Оформлення пояснювальної записки	травень 2026	виконано

Студент

Давидчук А. М.

Укладач: старший викладач

Васильєва М. Д.

АНОТАЦІЯ

У курсовій роботі розроблено програмну систему діалогової взаємодії з великими мовними моделями. Система призначена для створення та ведення чатів з локальною LLM, збереження історії повідомлень, пошуку по діалогах, експорту чату у Markdown, перемикання локальних моделей і використання LoRA/QLoRA-адаптерів.

Програмний продукт реалізовано мовою Python. Для web-інтерфейсу використано Flask, для збереження даних — PostgreSQL, для керування міграціями — Alembic, для запуску локальної мовної моделі — Hugging Face Transformers. Додатково реалізовано CLI-режим, потокову генерацію відповідей, кнопку зупинки генерації, generation presets, локальний model registry, підтримку математичних формул через KaTeX і підсвічування блоків коду.

У роботі сформовано функціональні та нефункціональні вимоги до системи, виконано аналіз предметної області, спроектовано багаторівневу архітектуру, ER-діаграму бази даних, UML-діаграми прецедентів, класів і послідовності. Під час реалізації використано принципи ООП, SOLID, розділення відповідальності, Repository/Storage Adapter, Service Layer та Adapter.

Працездатність системи перевірено автоматизованими тестами. Повний набір містить 105 успішних тестів, а покриття production-коду, отримане за допомогою `coverage.py`, становить 77%.

Ключові слова: велика мовна модель, LLM, Flask, PostgreSQL, Transformers, LoRA, QLoRA, web-застосунок, CLI, streaming generation, тестування.

ТЕХНІЧНЕ ЗАВДАННЯ

1. Найменування та область використання

Найменування розробки: система діалогової взаємодії з великими мовними моделями з web- та консольним інтерфейсами. Система може використовуватися як навчальний програмний продукт для демонстрації принципів побудови LLM-застосунків, роботи з локальними мовними моделями, організації persistence-рівня, web-інтерфейсу та автоматизованого тестування.

2. Підстава для розробки

Підставою для розробки є завдання на виконання курсової роботи з дисципліни «Інженерія програмного забезпечення» для спеціальності 123 «Комп'ютерна інженерія».

3. Мета та призначення розробки

Метою роботи є розробка програмної системи, яка забезпечує користувачу можливість вести діалог з локальною великою мовною моделлю, зберігати історію чатів, керувати моделями під час виконання програми та перевіряти роботу системи автоматизованими тестами.

Призначення системи:

- створення, перегляд, перейменування та видалення чатів;
- надсилання повідомлень і отримання відповідей від локальної LLM;
- збереження історії діалогів у PostgreSQL;
- робота через web-інтерфейс і CLI;
- потокове відображення відповіді моделі;
- зупинка активної генерації без очікування повного завершення відповіді;
- пошук повідомлень і експорт діалогу у Markdown;
- відображення Markdown, блоків коду та математичних формул;
- перемикання локальних моделей і використання LoRA/QLoRA-адаптерів;
- вибір preset-ів генерації та перегляд стану активної моделі;
- тестування ключових сценаріїв роботи системи.

4. Джерела розробки

Джерелами розробки є документація Python, Flask, PostgreSQL, Alembic, Hugging Face Transformers, PEFT, pytest, KaTeX, а також матеріали з архітектури програмного забезпечення, UML-моделювання та тестування програмних систем.

5. Технічні вимоги

Вимоги до програмного продукту:

- система повинна мати web-інтерфейс для роботи з чатами;
- система повинна підтримувати CLI-режим;
- система повинна зберігати історію чатів і повідомлень;
- система повинна підтримувати локальний LLM-backend;
- система повинна дозволяти перемикати локальні моделі під час роботи;
- система повинна підтримувати streaming generation;
- система повинна дозволяти зупиняти активну генерацію;
- система повинна підтримувати пошук повідомлень у чатах;
- система повинна підтримувати експорт діалогу у Markdown;
- система повинна відображати Markdown, блоки коду та математичні формули;
- система повинна показувати активний backend, модель, adapter і preset генерації;
- система повинна підтримувати generation presets;
- система повинна підтримувати підключення LoRA/QLoRA-адаптерів через конфігураційний файл;
- система повинна контрольовано повідомляти про помилки завантаження моделі або adapter-a;
- система повинна мати автоматизовані тести для основної бізнес-логіки.

Вимоги до програмного забезпечення:

- Python 3.10 або вище;
- PostgreSQL;
- Flask;
- Hugging Face Transformers;
- PEFT для підключення LoRA/QLoRA-адаптерів;
- Alembic;
- pytest і coverage.py;

- KaTeX для відображення математичних формул у web-інтерфейсі.

Вимоги до апаратної частини:

- персональний комп'ютер або ноутбук, здатний запускати Python-застосунок;
- достатній обсяг оперативної пам'яті для запуску обраної локальної LLM;
- достатній обсяг дискового простору для збереження локальних моделей і adapter-ів;
- GPU є бажаним для швидшої генерації, але система може працювати і на CPU з малими моделями.

6. Етапи розробки

Основними етапами розробки є отримання теми, вивчення джерел, формування вимог, проєктування архітектури та UML-діаграм, реалізація програмного продукту, тестування, оформлення пояснювальної записки та підготовка до захисту.

Зміст

ЗАВДАННЯ НА КУРСОВУ РОБОТУ	1
АНОТАЦІЯ	3
ТЕХНІЧНЕ ЗАВДАННЯ	4
ВСТУП	8
1 АНАЛІТИЧНИЙ ОГЛЯД ПРЕДМЕТНОЇ ОБЛАСТІ	10
1.1. Характеристика предметної області діалогових систем на основі великих мовних моделей	10
1.2. Аналіз підходів до побудови програмних систем для взаємодії з LLM	10
1.3. Огляд існуючих аналогів і подібних програмних рішень	11
1.4. Порівняльний аналіз існуючих рішень	11
1.5. Недоліки існуючих рішень та обґрунтування необхідності власної розробки .	13
1.6. Постановка задачі розробки програмної системи	13
2 ПРОЄКТУВАННЯ СИСТЕМИ ДІАЛОГОВОЇ ВЗАЄМОДІЇ З LLM	14
2.1. Вибір архітектури програмної системи	14
2.2. Формування функціональних і нефункціональних вимог	15
2.3. Матриця трасування вимог	17
2.4. Проєктування основних функціональних модулів	18
2.5. Проєктування структури бази даних	19
2.6. UML-діаграма прецедентів	21
2.7. UML-діаграма класів	22

2.8. UML-діаграма послідовності	23
2.9. Опис логічної структури системи	23
3 РЕАЛІЗАЦІЯ, ТЕСТУВАННЯ ТА ЕКСПЛУАТАЦІЯ ПРОГРАМНОЇ СИСТЕМИ	24
3.1. Загальна структура програмної реалізації	24
3.2. Реалізація доменної логіки	24
3.3. Застосовані принципи проєктування та патерни	25
3.4. Реалізація web-інтерфейсу	26
3.5. Реалізація CLI-режиму	27
3.6. Реалізація сховища даних	28
3.7. Реалізація LLM-рівня	28
3.8. Runtime-керування моделлю та потокова генерація	29
3.9. Конфігурація та локальні моделі	29
3.10. Тестування програмної системи	30
3.11. Порядок запуску та експлуатації	33
ВИСНОВКИ	34
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ	35
ДОДАТКИ	37
Додаток А. Структура репозиторію програмного продукту	37
Додаток Б. Фрагменти модульних тестів	37
Додаток В. Результати покриття коду тестами	39
Додаток Г. Приклади функціонального тестування web/API	40

ВСТУП

Сучасний розвиток програмних систем дедалі частіше пов'язаний з інтеграцією засобів штучного інтелекту, зокрема великих мовних моделей. Такі моделі використовуються для побудови діалогових асистентів, автоматизованого пояснення навчального матеріалу, генерації програмного коду, аналізу текстів та підтримки користувача під час роботи з інформаційними системами. Водночас практичне використання LLM потребує не лише самої моделі, а й прикладного програмного середовища, яке забезпечує введення запитів, збереження історії діалогів, відображення відповідей, керування параметрами генерації та обробку помилок.

Актуальність теми курсової роботи зумовлена необхідністю створення навчальної програмної системи, яка демонструє повний цикл взаємодії користувача з локальною великою мовною моделлю. На відміну від готових хмарних сервісів, власна реалізація дозволяє дослідити архітектуру такої системи, організацію web- та CLI-інтерфейсів, роботу зі сховищем даних, інтеграцію локальної pretrained-моделі, потокову генерацію відповідей та розширення через LoRA/QLoRA-адаптери.

Метою курсової роботи є розробка програмної системи діалогової взаємодії з великими мовними моделями з підтримкою web- та консольного інтерфейсів, збереженням історії діалогів у PostgreSQL та можливістю запуску локальних моделей через Hugging Face Transformers.

Для досягнення поставленої мети необхідно виконати такі завдання:

1. проаналізувати предметну область діалогових систем на основі LLM;
2. розглянути існуючі рішення та визначити їхні переваги й обмеження;
3. сформулювати функціональні та нефункціональні вимоги до системи;
4. спроектувати архітектуру програмного продукту та структуру бази даних;
5. реалізувати web-інтерфейс для роботи з чатами та повідомленнями;
6. реалізувати CLI-режим роботи;
7. організувати збереження даних у PostgreSQL з використанням міграцій;
8. інтегрувати локальний LLM-backend на основі Transformers;
9. реалізувати runtime-перемикання моделей, streaming generation, зупинку генерації та підтримку LoRA/QLoRA-адаптерів;
10. виконати тестування основних модулів системи.

Об'єктом дослідження є процес побудови програмних систем для діалогової взаємодії користувача з великими мовними моделями.

Предметом дослідження є архітектурні, програмні та інтерфейсні засоби реалізації локальної LLM-системи з web- та CLI-режимами, базою даних PostgreSQL і локальним inference-backend.

Практичне значення роботи полягає у створенні працездатного програмного продукту, який може використовуватися як навчальний приклад побудови модульної LLM-системи, а також як основа для подальших експериментів з локальними моделями, адаптерами, параметрами генерації та розширенням функціональності.

РОЗДІЛ 1. АНАЛІТИЧНИЙ ОГЛЯД ПРЕДМЕТНОЇ ОБЛАСТІ

1.1. Характеристика предметної області діалогових систем на основі великих мовних моделей

Сучасні програмні системи дедалі частіше використовують засоби штучного інтелекту для автоматизованої обробки текстової інформації, підтримки користувача, генерації відповідей, пояснення навчального матеріалу, написання програмного коду та організації діалогової взаємодії. Одним із найбільш поширених напрямів розвитку таких систем є використання великих мовних моделей, або Large Language Models (LLM).

Великі мовні моделі — це програмні компоненти, побудовані на основі нейронних мереж, які здатні обробляти природну мову, аналізувати контекст запиту та генерувати текстові відповіді. У прикладних системах LLM зазвичай не використовуються ізольовано: навколо моделі створюється програмна інфраструктура, яка забезпечує введення повідомлень користувача, передавання запиту до моделі, отримання відповіді, збереження історії діалогу та відображення результату в зручному інтерфейсі.

Предметна область даної курсової роботи охоплює розробку програмної системи для діалогової взаємодії з великою мовною моделлю. Така система повинна забезпечувати користувачу можливість створювати окремі діалоги, надсилати повідомлення, отримувати відповіді від мовної моделі, переглядати історію листування та працювати з програмою як через веб-інтерфейс, так і через консольний режим.

Особливістю обраної предметної області є поєднання кількох напрямів програмної інженерії:

- побудова користувацького інтерфейсу для діалогової взаємодії;
- організація бізнес-логіки керування чатами та повідомленнями;
- інтеграція pretrained-моделі через окремий LLM-backend;
- збереження історії діалогів у реляційній базі даних PostgreSQL;
- створення модульної архітектури, що дозволяє надалі підключати інші моделі або адаптери;
- забезпечення тестованості основних компонентів системи.

Розроблювана система орієнтована на локальне або навчально-дослідницьке використання. На відміну від багатьох хмарних сервісів, вона передбачає можливість запуску на персональному комп'ютері, використання локально завантажених pretrained-моделей та контроль над структурою програмного продукту.

1.2. Аналіз підходів до побудови програмних систем для взаємодії з LLM

Побудова LLM-системи може виконуватися різними способами. Найпростішим варіантом є використання готового хмарного сервісу, де користувач взаємодіє з моделлю через

браузер або API. Такий підхід мінімізує складність розгортання, але зменшує контроль над моделлю, способом збереження даних і внутрішньою архітектурою.

Інший підхід полягає у використанні локальних інструментів для запуску open-source або open-weight моделей. У цьому випадку модель завантажується на комп'ютер користувача, а генерація відповідей виконується локально. Перевагою такого підходу є більший контроль над даними та можливість експериментувати з різними моделями. Недоліком є залежність від апаратних ресурсів комп'ютера.

Третій підхід — створення власної прикладної системи, яка інтегрує LLM як один із модулів. У цьому випадку мовна модель є не самостійним продуктом, а частиною програмної архітектури. Саме цей підхід обрано в даній курсовій роботі. Він дозволяє відокремити користувацький інтерфейс, бізнес-логіку, сховище даних і LLM-backend.

1.3. Огляд існуючих аналогів і подібних програмних рішень

Для визначення доцільності розробки власної системи було розглянуто декілька існуючих програмних рішень, пов'язаних із діалоговою взаємодією з LLM: ChatGPT, Claude, Google Gemini, LM Studio, Ollama та Open WebUI.

ChatGPT, Claude та Google Gemini є хмарними діалоговими сервісами, які надають користувачу зручний web-інтерфейс і високу якість відповідей. Однак такі системи не дають повного контролю над моделлю, способом збереження історії та внутрішньою архітектурою.

LM Studio та Ollama орієнтовані на локальний запуск моделей. Вони спрощують роботу з pretrained-моделями, але не є навчальними програмними системами, у яких користувач самостійно проєктує web-рівень, CLI-рівень, бізнес-логіку, сховище даних і LLM-backend.

Open WebUI є self-hosted web-інтерфейсом для взаємодії з LLM-backend-ами. Це функціонально близьке рішення, однак воно є готовою платформою, тоді як у даній курсовій роботі важливим є проєктування та реалізація власної модульної системи.

1.4. Порівняльний аналіз існуючих рішень

Порівняльний аналіз аналогів наведено в таблиці 1.1.

Таблиця 1.1 – Порівняння існуючих рішень для діалогової взаємодії з LLM

Рішення	Тип системи	Локальний запуск	Web	CLI	Власна БД	Навчальне розширення
ChatGPT	Хмарний сервіс	Ні	Так	Ні	Ні	Обмежене
Claude	Хмарний сервіс	Ні	Так	Ні	Ні	Обмежене
Google Gemini	Хмарний сервіс	Ні	Так	Ні	Ні	Обмежене
LM Studio	Локальний застосунок	Так	Так	Частково	Обмежено	Обмежене
Ollama	Локальний LLM-runner	Так	Ні / через сторонні UI	Так	Ні	Середнє
Open WebUI	Self-hosted web UI	Через backend	Так	Ні	Так	Середнє
Розроблювана система	Навчальна модульна LLM-система	Так	Так	Так	Так, PostgreSQL	Високе

1.5. Недоліки існуючих рішень та обґрунтування необхідності власної розробки

Аналіз аналогів показує, що існуючі рішення мають низку обмежень у контексті навчальної розробки LLM-системи. Хмарні сервіси не дозволяють повністю контролювати модель, механізм її запуску та структуру збереження діалогів. Локальні інструменти для запуску моделей часто зосереджені саме на інференсі, але не завжди забезпечують навчально корисну архітектуру з чітким поділом на web-рівень, CLI-рівень, бізнес-логіку, storage-рівень і LLM-рівень.

Тому розробка власної системи є обґрунтованою. Вона дозволяє реалізувати програмний продукт, який поєднує web-інтерфейс, консольний режим, збереження історії в PostgreSQL, LLM-backend із підтримкою локальної pretrained-моделі, runtime-керування моделлю, потокову генерацію відповідей та основу для подальшого розширення через LoRA/QLoRA-адаптацію.

1.6. Постановка задачі розробки програмної системи

Метою розробки є створення програмної системи для діалогової взаємодії з великою мовною моделлю з підтримкою web- та консольного інтерфейсів.

Для досягнення поставленої мети необхідно розв'язати такі задачі:

1. Розробити структуру програмного застосунку з головним файлом запуску `app.py`.
2. Реалізувати web-інтерфейс для створення, вибору, перейменування та видалення чатів.
3. Реалізувати консольний режим роботи для взаємодії з чатами через командний рядок.
4. Реалізувати модуль керування чатами та повідомленнями.
5. Організувати збереження історії чатів і повідомлень у PostgreSQL.
6. Реалізувати LLM-backend із можливістю використання pretrained-моделі через Hugging Face Transformers.
7. Забезпечити підтримку локально завантаженої instruction-моделі.
8. Додати відображення Markdown, блоків коду та математичних формул у web-інтерфейсі.
9. Реалізувати автоматизовані тести для основних компонентів системи.
10. Реалізувати підтримку LoRA/QLoRA-адаптерів для локальних моделей.

РОЗДІЛ 2. ПРОЄКТУВАННЯ СИСТЕМИ ДІАЛОГОВОЇ ВЗАЄМОДІЇ З LLM

2.1. Вибір архітектури програмної системи

Для розробки системи діалогової взаємодії з LLM обрано модульну багаторівневу архітектуру. Такий підхід дозволяє розділити відповідальність між окремими компонентами системи та забезпечити можливість подальшого розширення без істотної зміни існуючого коду.

Загальна архітектура системи складається з таких рівнів:

- рівень користувацької взаємодії;
- рівень бізнес-логіки;
- рівень збереження даних;
- рівень взаємодії з мовною моделлю;
- рівень конфігурації та допоміжних сервісів.

Рівень користувацької взаємодії представлений web-інтерфейсом і CLI-режимом. Рівень бізнес-логіки реалізовано у компоненті `ChatManager`. Рівень збереження даних реалізовано через `PostgresStorage`, а рівень взаємодії з мовною моделлю — через `BaseLLMService`, `TransformersLLMService`, `UnavailableLLMService` та `LLMRuntime`.

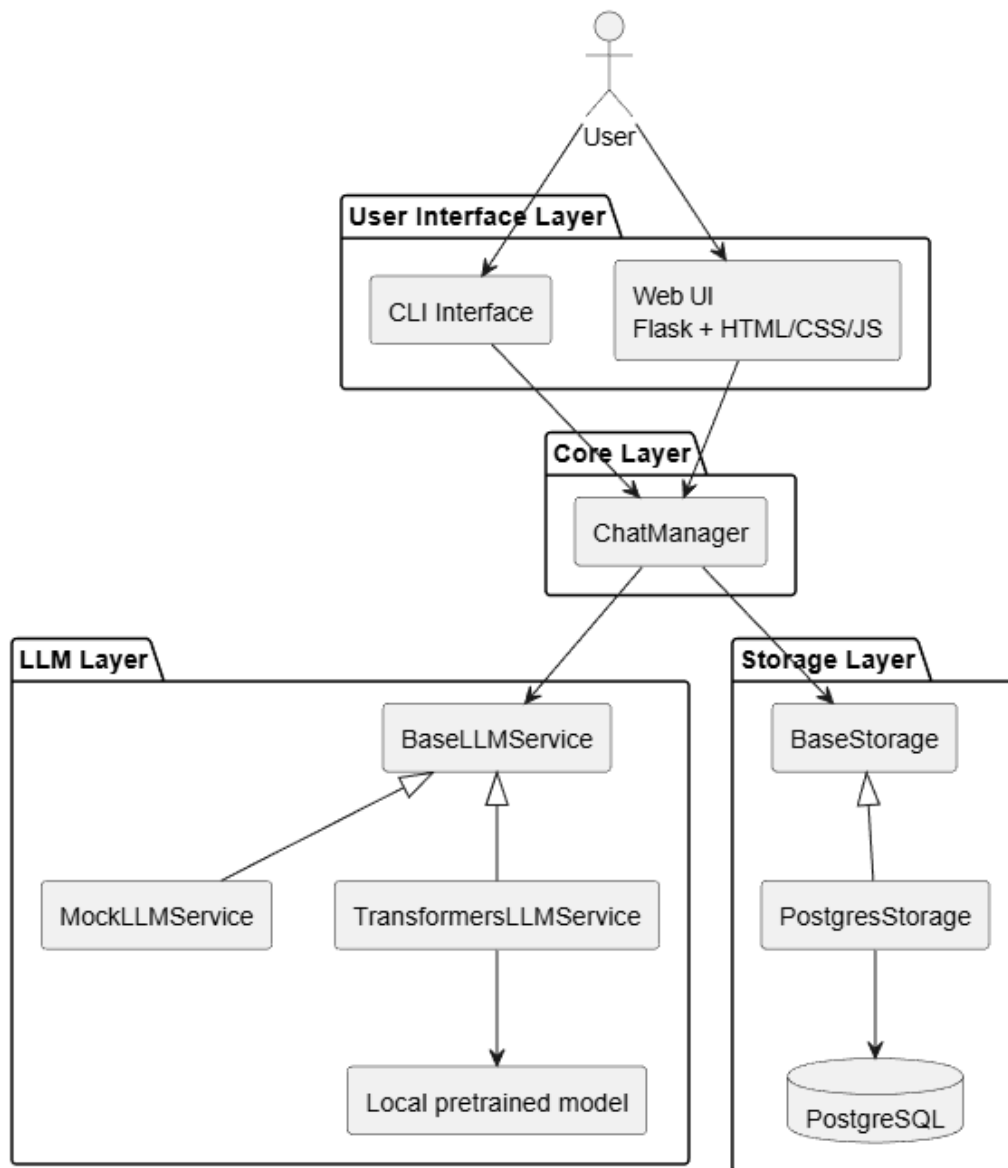


Рисунок 2.1 – Загальна архітектура системи

2.2. Формування функціональних і нефункціональних вимог

На основі технічного завдання та аналізу предметної області сформовано функціональні та нефункціональні вимоги до системи.

Таблиця 2.1 – Функціональні вимоги до системи

Код	Вимога
F1	Система повинна запускатися з головного файла <code>app.py</code> .
F2	Система повинна підтримувати web-режим роботи.
F3	Система повинна підтримувати CLI-режим роботи через параметр <code>-t</code> .
F4	Користувач повинен мати можливість створювати новий чат.
F5	Користувач повинен мати можливість відкривати наявний чат.
F6	Користувач повинен мати можливість перейменувати чат.

Код	Вимога
F7	Користувач повинен мати можливість видаляти чат.
F8	Користувач повинен мати можливість надсилати повідомлення.
F9	Система повинна генерувати відповідь через підключений LLM-backend.
F10	Система повинна зберігати історію чатів і повідомлень у PostgreSQL.
F11	Система повинна коректно повідомляти про помилки завантаження моделі.
F12	Система повинна підтримувати Transformers-backend для локальної pretrained-моделі.
F13	Система повинна відображати Markdown, блоки коду та математичні формули у відповідях асистента.
F14	Система повинна показувати активний LLM-backend, назву моделі та preset генерації.
F15	Система повинна підтримувати runtime-перемикання та вивантаження локальних моделей.
F16	Система повинна підтримувати streaming generation і зупинку активної генерації.
F17	Система повинна підтримувати підключення PEFT LoRA/QLoRA-адаптерів.
F18	Система повинна підтримувати пошук повідомлень і експорт діалогу у Markdown.

Таблиця 2.2 – Нефункціональні вимоги до системи

Код	Вимога
NF1	Архітектура системи повинна бути модульною: web-рівень, CLI-рівень, доменна логіка, сховище даних, LLM-рівень і конфігурація мають бути винесені в окремі модулі.
NF2	Компоненти системи повинні мати слабке зв'язування через абстракції <code>BaseStorage</code> і <code>BaseLLMService</code> ; доменна логіка не повинна напряму залежати від Flask, PostgreSQL або Transformers.
NF3	Система повинна бути придатною до подальшого розширення: додавання нового storage-backend, LLM-backend або конфігурації моделі не повинно вимагати зміни основної логіки <code>ChatManager</code> .
NF4	Основна бізнес-логіка повинна бути покрита автоматизованими тестами на рівні не менше 70% production-коду; фактичне покриття становить 77%, набір тестів містить 105 перевірок.

Код	Вимога
NF5	Система повинна коректно обробляти помилки користувача: порожні повідомлення, некоректні або дубльовані назви чатів, відсутні моделі та помилки адаптерів мають повертати контрольоване повідомлення, а не аварійне завершення.
NF6	Web-інтерфейс повинен бути зручним і зрозумілим: користувач має бачити активний backend, модель, adapter, preset генерації, стан завантаження моделі, кнопку зупинки генерації та кнопки копіювання для блоків коду.
NF7	Дані чатів повинні зберігатися у структурованому вигляді в таблицях <code>chats</code> і <code>messages</code> з ідентифікаторами чатів, ролями повідомлень, timestamp-ами та порядковим номером повідомлення.
NF8	Конфігурація системи повинна виконуватися через змінні середовища або <code>.env</code> файл; паролі, локальні моделі та адаптери не повинні додаватися до Git-репозиторію.
NF9	Система не повинна жорстко залежати від конкретної мовної моделі: доступні моделі мають визначатися через локальний каталог <code>models/</code> , <code>model_config.json</code> , runtime-перемикання та generation presets.
NF10	Система повинна підтримувати роботу на персональному комп'ютері з Python 3.10 або вище, PostgreSQL і локальними model artifacts; для великих моделей може використовуватися GPU, але малі моделі можуть запускатися на CPU.

2.3. Матриця трасування вимог

Для підтвердження повноти реалізації сформовано матрицю трасування, яка пов'язує вимоги з програмними модулями та способом перевірки. Такий підхід дозволяє показати, що функціональні й нефункціональні вимоги не є декларативними, а мають відповідні елементи реалізації та тестування.

Таблиця 2.3 – Матриця трасування вимог

Вимога	Модуль реалізації	Перевірка
F1–F3	<code>app.py</code> , <code>src/web</code> , <code>src/cli</code>	Перевірка запуску web-режиму, CLI-режиму та тестування базових web routes.
F4–F8	<code>ChatManager</code> , web endpoints, storage-рівень	Тести <code>chat_manager</code> і <code>web_app</code> : створення, відкриття, перейменування, видалення чатів і надсилання повідомлень.

Вимога	Модуль реалізації	Перевірка
F9–F12	BaseLLMService, TransformersLLMService, UnavailableLLMService, LLMRuntime	Тести runtime-станів, побудови prompt-ів, fallback-поведінки та обробки помилок завантаження моделі.
F13	HTML-шаблони, <code>script.js</code> , Markdown/KaTeX renderer	Ручна browser-перевірка Markdown, code blocks і формул, а також regression-перевірки web-відображення.
F14–F17	LLMRuntime, <code>model_registry</code> , <code>model_config.json</code> , generati- on presets, PEFT adapter loading	Тести <code>llm_runtime</code> , <code>model_registry</code> , <code>config_presets</code> , <code>transformers_prompting</code> .
F18	ChatManager, routes пошуку та експорту	Тести пошуку повідомлень і експорту Markdown у групах <code>chat_manager</code> та <code>web_app</code> .
NF1–NF3	Поділ на <code>src/core</code> , <code>src/storage</code> , <code>src/llm</code> , <code>src/web</code> , <code>src/cli</code>	Перевірка структури коду, використан- ня абстракцій і можливість підстановки fake-реалізацій у тестах.
NF4	Набір автоматизованих те- стів, <code>coverage.py</code>	105 успішних тестів, production coverage 77%.
NF5–NF8	Валідація у ChatManager, web routes, storage factory, <code>src/config.py</code>	Тести помилкових сценаріїв, конфігура- ції, Alembic-міграцій і storage factory.
NF9–NF10	<code>model_registry</code> , runtime- перемикання, локальні model artifacts	Перевірка сканування моделей, читан- ня <code>model_config.json</code> , перемикання і ви- вантаження моделей.

2.4. Проектування основних функціональних модулів

Програмну систему поділено на такі основні модулі: web-інтерфейс, CLI-інтерфейс, модуль керування чатами, LLM-backend, runtime-керування моделлю, модуль збереження даних і конфігураційний модуль.

Модуль web-інтерфейсу відповідає за взаємодію користувача із системою через браузер. CLI-модуль забезпечує альтернативний текстовий режим роботи. Модуль ChatManager координує операції між інтерфейсами користувача, сховищем даних і LLM-сервісом. LLM-модуль реалізує генерацію відповідей через локальну pretrained-модель. Runtime-модуль керує станом моделі, завантаженням, вивантаженням, streaming-генерацією та зупинкою генерації. Модуль збереження даних використовує PostgreSQL для збереження чатів і повідомлень.

2.5. Проектування структури бази даних

Для збереження історії діалогів використовується реляційна база даних PostgreSQL. Структура бази даних складається з двох основних таблиць: `chats` і `messages`.

Таблиця 2.4 – Структура таблиці `chats`

Поле	Тип	Опис
<code>id</code>	TEXT	Унікальний ідентифікатор чату
<code>title</code>	TEXT	Назва чату
<code>created_at</code>	TIMESTAMPTZ	Дата і час створення чату
<code>updated_at</code>	TIMESTAMPTZ	Дата і час останнього оновлення чату

Таблиця 2.5 – Структура таблиці `messages`

Поле	Тип	Опис
<code>id</code>	BIGSERIAL	Унікальний ідентифікатор повідомлення
<code>chat_id</code>	TEXT	Ідентифікатор чату, до якого належить повідомлення
<code>role</code>	TEXT	Роль автора повідомлення: <code>user</code> або <code>assistant</code>
<code>content</code>	TEXT	Текст повідомлення
<code>timestamp</code>	TIMESTAMPTZ	Дата і час створення повідомлення
<code>position</code>	INTEGER	Порядковий номер повідомлення в чаті

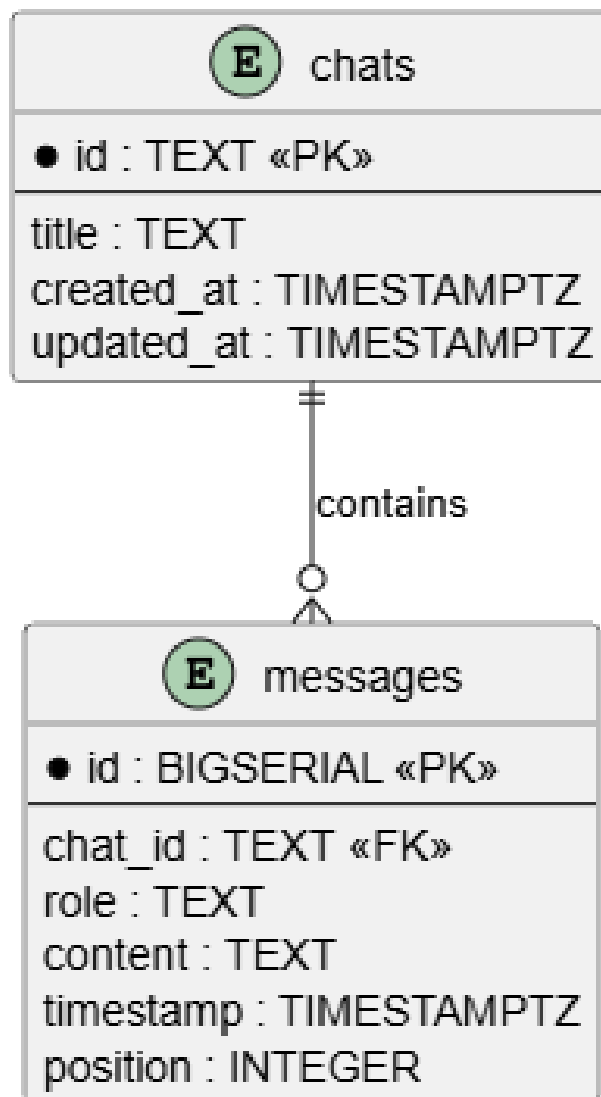


Рисунок 2.2 – ER-діаграма бази даних

2.6. UML-діаграма прецедентів



Рисунок 2.3 – UML-діаграма прецедентів системи

2.7. UML-діаграма класів

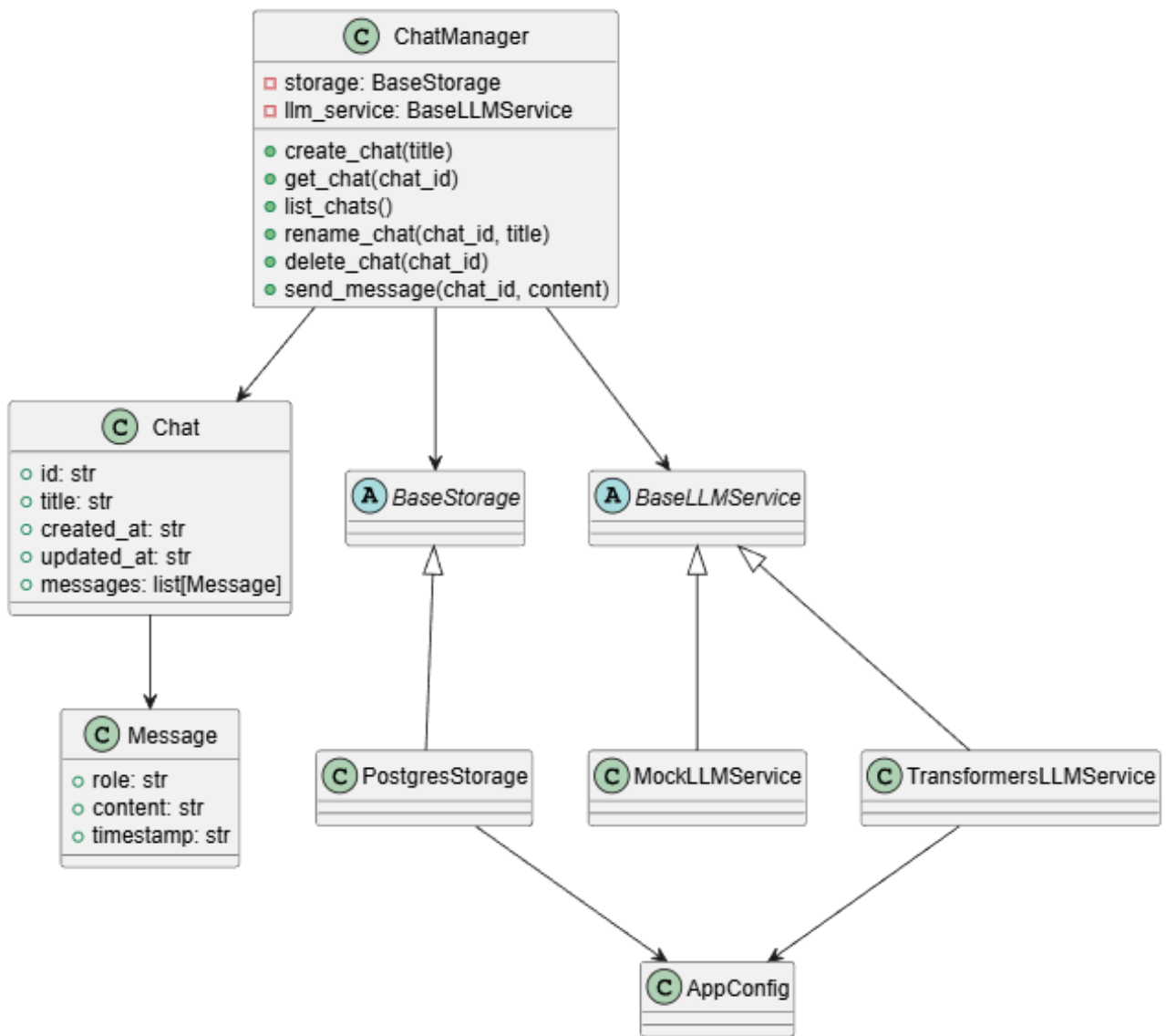


Рисунок 2.4 – UML-діаграма класів системи

2.8. UML-діаграма послідовності

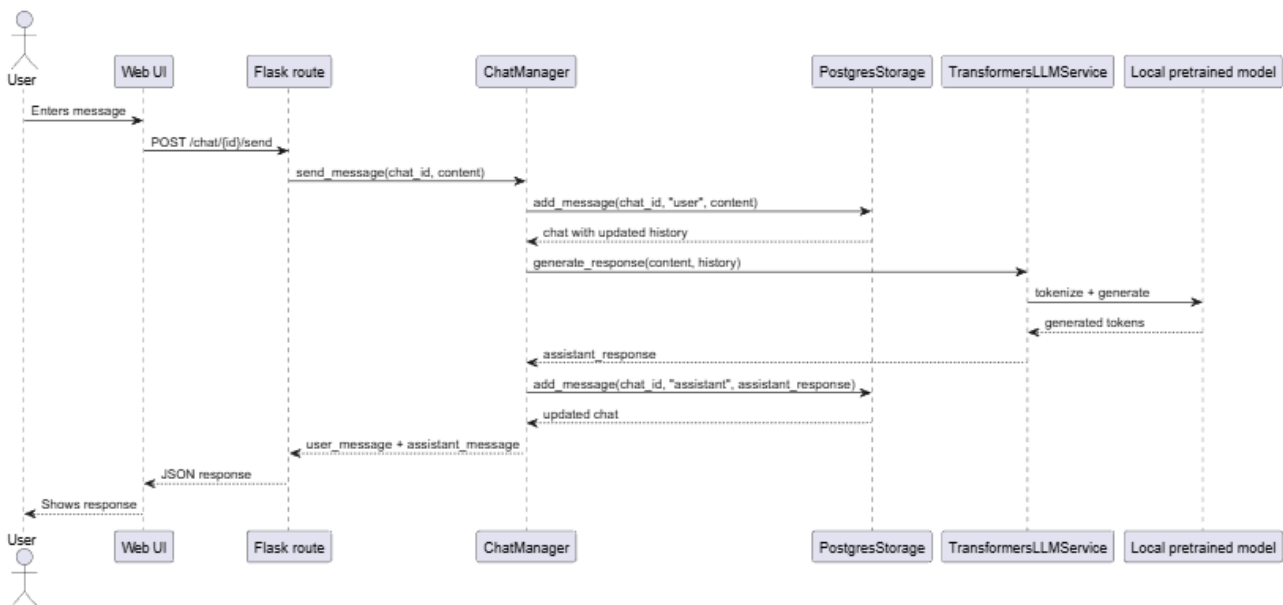


Рисунок 2.5 – UML-діаграма послідовності надсилання повідомлення

2.9. Опис логічної структури системи

Логічна структура системи побудована так, щоб окремі частини програмного продукту мали чітко визначені обов'язки. Web-інтерфейс і CLI-інтерфейс не працюють безпосередньо з базою даних або моделлю. Вони звертаються до **ChatManager**, який координує виконання операцій.

ChatManager не залежить від конкретної реалізації LLM-рівня. Для генерації відповідей використовується інтерфейс **BaseLLMService**. Для збереження даних у поточній реалізації використовується **PostgresStorage**, що працює з PostgreSQL.

PostgreSQL використовується як основне сховище даних. Це дозволяє зберігати історію діалогів у структурованому вигляді, підтримувати зв'язок між чатами й повідомленнями та надалі розширювати схему бази даних.

LLM-рівень підтримує Transformers-режим, який дозволяє підключати pretrained-модель, завантажену в локальну директорію `models/`. Якщо модель не може бути завантажена, використовується **UnavailableLLMService**, що зберігає інформацію про помилку та дозволяє відобразити її у web-інтерфейсі. Система реалізує потокову генерацію відповідей, зупинку генерації, runtime-перемикання моделей без перезапуску застосунку та підключення LoRA/QLoRA-адаптерів.

Конфігурація системи виконується через змінні середовища або `.env` файл. Це дозволяє змінювати LLM-backend, модель, preset генерації та параметри підключення до PostgreSQL без зміни вихідного коду.

Таким чином, обрана структура забезпечує модульність, розширюваність, зручність тестування та відповідність поставленим вимогам до курсового проєкту.

РОЗДІЛ 3. РЕАЛІЗАЦІЯ, ТЕСТУВАННЯ ТА ЕКСПЛУАТАЦІЯ ПРОГРАМНОЇ СИСТЕМИ

3.1. Загальна структура програмної реалізації

Програмну систему реалізовано мовою Python з використанням Flask для web-рівня, PostgreSQL для збереження даних, Alembic для керування міграціями бази даних та Hugging Face Transformers для запуску локальної мовної моделі. Вихідний код розділено на окремі модулі відповідно до їхньої відповідальності.

Головним файлом запуску є `app.py`. Він аналізує параметри командного рядка і запускає один із двох режимів роботи:

- web-режим, у якому створюється Flask-застосунок;
- CLI-режим, який активується параметром `-t` або `-terminal`.

Основні каталоги програмної системи наведено у таблиці 3.1.

Таблиця 3.1 – Структура програмної реалізації

Каталог або файл	Призначення
<code>app.py</code>	Головна точка входу, вибір web- або CLI-режиму.
<code>src/core</code>	Доменні моделі та менеджер чатів.
<code>src/web</code>	Flask routes, HTML-шаблони, CSS і JavaScript web-інтерфейсу.
<code>src/cli</code>	Реалізація консольного режиму.
<code>src/storage</code>	Абстракція сховища та PostgreSQL-реалізація.
<code>src/llm</code>	Інтерфейс LLM-сервісу, runtime-керування моделлю та Transformers-backend.
<code>migrations</code>	Alembic-міграції схеми бази даних.
<code>scripts</code>	Допоміжні скрипти для завантаження моделей, адаптерів і експериментального fine-tuning.
<code>tests</code>	Автоматизовані тести системи.

Такий поділ дозволяє відокремити інтерфейс користувача від бізнес-логіки, сховища даних та LLM-рівня. Це спрощує тестування, підтримку та подальший розвиток системи.

3.2. Реалізація доменної логіки

Доменний рівень системи реалізовано у модулі `src/core`. Основними сутностями є `Chat`, `Message` та `SearchResult`. Вони описують чат, окреме повідомлення та результат пошуку відповідно.

Клас `ChatManager` є центральним сервісом прикладної логіки. Він не залежить безпосередньо від Flask, PostgreSQL або Transformers, а працює через абстракції `BaseStorage`

та `BaseLLMService`. Завдяки цьому один і той самий менеджер використовується як у web-інтерфейсі, так і в CLI-режимі.

Основні можливості `ChatManager`:

- створення чатів з унікальними назвами;
- перевірка назв чатів на порожнє значення, довжину та дублювання;
- отримання списку чатів;
- додавання повідомлень користувача й асистента;
- генерація відповіді через підключений LLM-сервіс;
- streaming-генерація з подальшим збереженням фінальної відповіді;
- пошук повідомлень;
- перейменування та видалення чатів.

При надсиланні повідомлення `ChatManager` спочатку зберігає повідомлення користувача, формує історію діалогу, передає її до LLM-сервісу, отримує відповідь і зберігає її як повідомлення асистента. Такий підхід забезпечує цілісність історії діалогу та дозволяє моделі враховувати контекст поточного чату.

3.3. Застосовані принципи проектування та патерни

Під час реалізації системи було використано об'єктно-орієнтований підхід з розділенням відповідальності між окремими класами та модулями. Доменні сутності `Chat`, `Message` і `SearchResult` описують основні об'єкти предметної області, а сервіс `ChatManager` інкапсулює бізнес-логіку роботи з діалогами.

Основні принципи проектування, використані у системі:

- **Single Responsibility.** Кожен основний компонент має окрему зону відповідальності: `ChatManager` керує сценаріями чатів, `PostgresStorage` відповідає за збереження даних, `TransformersLLMService` інкапсулює роботу з локальною моделлю, а `LLMRuntime` керує станом завантаженої моделі.
- **Dependency Inversion.** Доменна логіка залежить не від конкретної PostgreSQL-реалізації або Transformers-backend, а від абстракцій `BaseStorage` і `BaseLLMService`. Завдяки цьому у тестах можна підставляти `InMemoryStorage` і `FakeLLMService`.
- **Open/Closed.** Система може бути розширена новими storage-backend, LLM-backend або конфігураціями моделей без переписування доменної логіки.
- **Separation of Concerns.** Web-рівень, CLI-рівень, бізнес-логіка, сховище даних, LLM-рівень і конфігурація винесені в окремі модулі.

- **Liskov Substitution.** Реалізації `PostgresStorage` і `InMemoryStorage` можуть використовуватися через спільний контракт `BaseStorage`, а `fake-`, `unavailable-` і `transformers-`сервіси — через контракт `BaseLLMService`.
- **Interface Segregation.** Зовнішні рівні системи працюють з обмеженими контрактами, потрібними саме для їхньої задачі: `storage-`рівень відповідає за дані, а `LLM-`рівень — за генерацію.

У програмі застосовано кілька практичних архітектурних патернів. Патерн **Repository/Storage Adapter** використано у рівні збереження даних: доменна логіка працює через `BaseStorage`, а конкретна реалізація `PostgresStorage` приховує SQL-запити та структуру таблиць. Патерн **Service Layer** реалізовано через `ChatManager`, який координує роботу між інтерфейсом, сховищем і LLM-сервісом. Патерн **Adapter** також використовується на LLM-рівні: `transformers-`сервіс приводить зовнішній API Hugging Face до внутрішнього інтерфейсу `BaseLLMService`.

Принцип **Inversion of Control** реалізовано через те, що ключові залежності створюються на рівні композиції застосунку, а не всередині доменної логіки. Web-застосунок і CLI-режим отримують готовий `ChatManager`, а сам `ChatManager` отримує `storage-` та `LLM-`сервіси через конструктор. Завдяки цьому у `production-`режимі можна використати `PostgreSQL-` і `Transformers-`реалізації, а у тестах — `in-memory` сховище та `fake LLM-`сервіс.

Принцип **Composition over Inheritance** проявляється у тому, що поведінка системи формується через поєднання незалежних компонентів, а не через глибоку ієрархію наслідування. Наприклад, `ChatManager` не наслідує `storage` або `LLM-`класи, а містить посилання на відповідні сервіси. `LLMRuntime` також не є спеціалізованою моделлю, а керує поточним LLM-сервісом як окремою залежністю.

Принципи **Loose Coupling** і **High Cohesion** забезпечуються тим, що кожен модуль має вузьку зону відповідальності та взаємодіє з іншими модулями через стабільні інтерфейси. Це дозволяє окремо змінювати `web-`інтерфейс, `storage-`рівень, `runtime-`керування моделлю або конфігурацію моделей без переписування всієї системи.

Окремо використано принцип конфігураційного розширення: локальні моделі та LoRA-адаптери не жорстко прописані в коді, а описуються через `model_config.json` і змінні середовища. Це спрощує зміну моделі без зміни програмної логіки та робить систему зручнішою для подальших експериментів.

3.4. Реалізація web-інтерфейсу

Web-інтерфейс реалізовано за допомогою Flask. Функція `create_app` створює застосунок, ініціалізує `ChatManager`, `LLMRuntime` та реєструє HTTP routes.

Основні маршрути `web-`застосунку:

- `GET /` — головна сторінка зі списком чатів;
- `POST /chats` — створення нового чату;

- GET /chat/<chat_id> — відкриття конкретного чату;
- POST /chat/<chat_id>/send — надсилання повідомлення без streaming;
- POST /chat/<chat_id>/stream — потокова генерація відповіді;
- POST /api/generation/stop — зупинка активної генерації;
- POST /chat/<chat_id>/rename — перейменування чату;
- POST або DELETE /chat/<chat_id>/delete — видалення чату;
- GET /chat/<chat_id>/export — експорт чату у Markdown;
- GET /api/model/status — отримання стану моделі;
- POST /api/model/switch — перемикання моделі;
- POST /api/model/unload — вивантаження моделі з пам'яті;
- GET /api/messages/search — пошук повідомлень;
- POST /api/generation-preset — зміна preset генерації.

Клієнтська частина реалізована у файлах `script.js` та `style.css`. JavaScript-код відповідає за надсилання повідомлень без перезавантаження сторінки, streaming-відображення відповіді, керування кнопкою зупинки генерації, пошук повідомлень, перейменування чатів, перемикання моделей та показ toast-повідомлень.

У web-інтерфейсі реалізовано відображення Markdown, блоків коду, кнопки копіювання коду та математичних формул. Для рендерингу формул використовується локально підключений KaTeX, що дозволяє відображати LaTeX-подібні формули без звернення до зовнішніх CDN.

3.5. Реалізація CLI-режиму

Консольний режим реалізовано у модулі `src/cli/cli_app.py`. Він використовує той самий `ChatManager`, що й web-інтерфейс, тому логіка створення чатів, вибору наявних чатів, збереження повідомлень і генерації відповідей залишається спільною.

CLI-режим надає користувачу просте текстове меню:

1. створити новий чат;
2. вибрати існуючий чат;
3. завершити роботу.

Після вибору чату користувач може вводити повідомлення у командному рядку. Команда `/exit` повертає до головного меню, а `/quit` завершує роботу програми. Наявність CLI-режиму підтверджує, що бізнес-логіка системи не залежить від web-рівня.

3.6. Реалізація сховища даних

Для збереження історії діалогів використовується PostgreSQL. Рівень збереження реалізовано через інтерфейс `BaseStorage` і конкретну реалізацію `PostgresStorage`. Такий підхід відповідає патерну Repository/Storage Adapter і приховує SQL-запити від доменної логіки.

У базі даних використовуються таблиці `chats` і `messages`. Таблиця `messages` має поле `position`, яке визначає порядок повідомлень у межах одного чату. Для підвищення ефективності запитів створено індекси за датою оновлення чатів, позицією повідомлень у чаті та `timestamp` повідомлень.

Схема бази даних керується через Alembic. Під час старту `PostgresStorage` виконує застосування актуальних міграцій. Це дозволяє уникнути ручного створення таблиць після первинного створення самої бази даних.

3.7. Реалізація LLM-рівня

LLM-рівень побудовано навколо абстрактного класу `BaseLLMService`. Він визначає методи для звичайної та потокової генерації:

- `generate_response`;
- `generate_response_stream`;
- `finalize_streamed_response`.

Основною реалізацією є `TransformersLLMService`. Вона завантажує tokenizer і модель через Hugging Face Transformers, обирає пристрій виконання, формує prompt, запускає генерацію та очищує отриману відповідь. Для instruction-моделей використовується chat template tokenizer-a, а якщо він недоступний, застосовується fallback-побудова plain prompt.

Для випадків, коли модель не завантажилася, використовується fallback LLM-сервіс. Він зберігає інформацію про помилку і дозволяє web-інтерфейсу показати коректний статус замість аварійного завершення застосунку.

Система підтримує generation presets:

- `precise` — більш сфокусована генерація;
- `balanced` — збалансований режим;
- `creative` — більш вільна генерація з більшою кількістю токенів.

Також реалізовано підтримку PEFT LoRA/QLoRA-адаптерів. У цьому випадку базова модель завантажується з каталогу `models/`, а adapter підключається через API бібліотеки PEFT. Налаштування комбінацій базових моделей і адаптерів виконується через конфігураційний файл моделей.

3.8. Runtime-керування моделлю та потокова генерація

Оскільки локальна LLM є ресурсом, який може займати значний обсяг оперативної або відеопам'яті, у системі реалізовано окремий компонент `LLMRuntime`. Він відповідає за стан моделі, перемикання, вивантаження, блокування конфліктних операцій та зупинку активної генерації.

Основні стани runtime:

- `ready` — модель готова до генерації;
- `loading` — виконується завантаження або перемикання моделі;
- `error` — модель не завантажилась через помилку;
- `not_loaded` — модель вивантажена.

Перемикання моделей виконується асинхронно, щоб web-застосунок не блокувався під час тривалого завантаження. Під час генерації заборонено перемикати або вивантажувати модель, оскільки це могло б призвести до некоректної роботи inference-процесу.

Для покращення користувацького досвіду реалізовано streaming generation. Відповідь моделі передається у браузер частинами через NDJSON-потік. Користувач бачить відповідь поступово, а не лише після завершення генерації. Крім того, реалізовано кнопку зупинки генерації, яка встановлює спеціальний `Event`, що перевіряється під час роботи Transformers.

Окремо реалізовано background generation jobs. Генерація виконується у фоні, а HTTP stream лише читає події з черги. Це підвищує надійність: навіть якщо клієнтський stream обірветься, backend може завершити генерацію і зберегти відповідь у чат. На рівні інтерфейсу під час активної генерації заблоковано перехід до інших чатів і сторінок, щоб користувач не втрачав контекст поточної відповіді.

3.9. Конфігурація та локальні моделі

Конфігурація системи зосереджена у файлі `src/config.py`. Параметри можуть задаватися через змінні середовища або файл `.env`. Основні параметри:

- `DATABASE_URL` або окремі параметри PostgreSQL;
- `LLM_BACKEND`;
- `MODEL_NAME`;
- `ADAPTER_PATH`;
- `GENERATION_PRESET`;
- `MAX_NEW_TOKENS`, `TEMPERATURE`, `TOP_P`;
- `DEVICE`;

- `SYSTEM_PROMPT`.

Web-інтерфейс автоматично сканує каталог `models/` і показує локально доступні моделі. Окремо файл `model_config.json` дозволяє описати наперед підготовлені конфігурації, зокрема поєднання базової моделі та LoRA-адаптера.

3.10. Тестування програмної системи

Для перевірки працездатності системи використано автоматизовані тести на базі `pytest`. Тести охоплюють як доменну логіку, так і `web endpoints`, `storage-рівень`, `LLM-runtime`, побудову `prompt-ів`, конфігурацію та допоміжні скрипти.

Станом на момент підготовки звіту повний набір тестів містить 105 успішних перевірок. Основні групи тестів наведено у таблиці 3.2.

Таблиця 3.2 – Групи автоматизованих тестів

Група тестів	Що перевіряється
<code>chat_manager</code>	Створення, перейменування, видалення чатів, додавання повідомлень, <code>streaming-логіка</code> , пошук.
<code>web_app</code>	<code>Flask endpoints</code> , <code>web-операції</code> , <code>streaming</code> , <code>stop generation</code> , <code>model switching</code> , <code>export</code> , <code>search</code> .
<code>llm_runtime</code>	<code>Runtime-стани моделі</code> , <code>async switch</code> , <code>unload</code> , <code>stop event</code> , помилки завантаження.
<code>transformers_prompting</code>	<code>Chat template</code> , <code>fallback prompt</code> , очищення відповіді, <code>identity guard</code> , валідація <code>adapter path</code> .
<code>model_registry</code>	Сканування локальних моделей і читання <code>model_config.json</code> .
<code>migrations</code> та <code>storage_factory</code>	<code>PostgreSQL URL</code> , <code>Alembic upgrade</code> , вимоги до конфігурації сховища.
<code>config_presets</code>	<code>Generation presets</code> і <code>parsing конфігураційних значень</code> .
<code>download_adapter_script</code>	Допоміжний скрипт завантаження <code>PEFT/LoRA-адаптерів</code> .

Для ізоляції тестів використовуються `InMemoryStorage` та `FakeLLMService`. Це дозволяє перевіряти бізнес-логіку без реальної `PostgreSQL`-бази та без завантаження важкої локальної моделі. `PostgreSQL integration tests` винесено в окрему групу і потребують наявності змінної `DATABASE_URL`.

Окремо було сформовано набір тестових сценаріїв, які відображають основні функціональні можливості та критичні точки системи. Приклади тестових сценаріїв наведено у таблиці 3.3.

Таблиця 3.3 – Тестові сценарії програмної системи

ID	Об'єкт тестування	Сценарій	Очікуваний результат	Ст.
TC-01	ChatManager	Створення нового чату з коректною назвою.	Чат створено, має унікальний ідентифікатор і доступний у списку чатів.	ОК
TC-02	ChatManager	Спроба створити чат з порожньою або дубльованою назвою.	Система повертає помилку валідації і не створює некоректний запис.	ОК
TC-03	ChatManager	Перейменування існуючого чату.	Назва оновлюється, дата зміни чату оновлюється, дублікати блокуються.	ОК
TC-04	ChatManager	Видалення чату разом з повідомленнями.	Чат видаляється зі сховища і більше не повертається у списку.	ОК
TC-05	ChatManager	Надсилання повідомлення користувача і отримання відповіді асистента.	Обидва повідомлення збережено у правильному порядку.	ОК
TC-06	Web endpoints	Створення, відкриття, перейменування та видалення чату через HTTP routes.	Web-запити повертають коректні статуси і змінюють стан системи.	ОК
TC-07	Web streaming	Надсилання повідомлення через streaming endpoint.	Відповідь повертається частинами, після завершення фінальний текст збережено в історії.	ОК
TC-08	Stop generation	Натискання кнопки зупинки під час активної генерації.	Runtime отримує stop event, генерація завершується контрольовано.	ОК
TC-09	Search	Пошук повідомлень за текстовим запитом.	Система повертає релевантні повідомлення з інформацією про чат і роль автора.	ОК

ID	Об'єкт тестування	Сценарій	Очікуваний результат	Ст.
TC-10	Export	Експорт історії чату у Markdown.	Користувач отримує Markdown-файл з повідомленнями та часовими мітками.	ОК
TC-11	LLMRuntime	Перемикання моделі під час неактивної генерації.	Runtime переходить у стан завантаження, після успіху показує нову активну модель.	ОК
TC-12	LLMRuntime	Спроба перемкнути або вивантажити модель під час генерації.	Операція блокується, щоб не порушити активний inference-процес.	ОК
TC-13	LLM service	Побудова prompt через chat template або fallback-формат.	Prompt формується з урахуванням системного повідомлення та історії діалогу.	ОК
TC-14	Model registry	Сканування каталогу локальних моделей і читання model_config.json.	У sidebar доступні локальні моделі та описані конфігурації з LoRA-адаптерами.	ОК
TC-15	Storage layer	Робота PostgresStorage і фабрики сховища.	Сховище коректно створює, читає, оновлює і видаляє чати та повідомлення.	ОК
TC-16	Alembic migrations	Перевірка застосування міграцій бази даних.	Схема PostgreSQL приводиться до актуального стану без ручного створення таблиць.	ОК
TC-17	Generation presets	Зміна preset генерації через API.	Параметри генерації оновлюються і використовуються під час наступних відповідей.	ОК
TC-18	Adapter loading	Перевірка конфігурації LoRA/QLoRA-адаптера.	Система валідує шлях до адаптера і підключає його до відповідної базової моделі.	ОК

Для оцінювання кількісного покриття було використано інструмент `coverage.py`. Повний запуск тестів виконувався командою:

```
python -m coverage run -m pytest
```

Після виконання тестів було сформовано звіт покриття для production-коду системи:

```
python -m coverage report --include="src/*,migrations/*,scripts/*"
```

Результат запуску: **105 тестів успішно пройдено**, загальне покриття production-коду становить **77%**. Це відповідає рекомендованому рівню покриття 70–80% для основної бізнес-логіки та ключових інтеграційних точок. Найвище покриття мають модулі доменної логіки, конфігурації, runtime-керування моделлю, registry локальних моделей та web-рівень. Нижче покриття має TransformersLLMService, оскільки частина його коду залежить від фактичного завантаження локальної LLM і GPU/CPU inference-середовища, що у unit-тестах ізолюється через fake-сервіси.

3.11. Порядок запуску та експлуатації

Для запуску системи необхідно встановити Python 3.10 або вище, створити віртуальне середовище, встановити залежності з `requirements.txt`, створити PostgreSQL-базу даних і налаштувати файл `.env`.

Приклад запуску web-режиму:

```
python app.py
```

Приклад запуску CLI-режиму:

```
python app.py -t
```

Приклад запуску тестів:

```
pytest
```

Для локальної роботи з instruction-моделлю рекомендовано використовувати модель Qwen2.5-1.5B-Instruct. Вона розміщується у каталозі локальних моделей `models/`. Моделі та адаптери є локальними артефактами і не додаються до Git.

Таким чином, реалізована система відповідає поставленим вимогам: підтримує web-і CLI-режими, зберігає історію у PostgreSQL, виконує генерацію через локальний LLM-backend, дозволяє перемикаати моделі, використовувати presets та LoRA-адаптери, а також має автоматизоване тестове покриття основних компонентів.

ВИСНОВКИ

У ході виконання курсової роботи було розроблено програмну систему діалогової взаємодії з великими мовними моделями з web- та консольним інтерфейсами. Система забезпечує створення й керування чатами, надсилання повідомлень, отримання відповідей від локальної LLM-моделі, збереження історії у PostgreSQL та роботу з локальними моделями через Hugging Face Transformers.

У першому розділі було проаналізовано предметну область LLM-систем, розглянуто основні підходи до побудови діалогових застосунків і виконано порівняння з існуючими рішеннями, такими як ChatGPT, Claude, Google Gemini, LM Studio, Ollama та Open WebUI. На основі аналізу обґрунтовано доцільність створення власної навчальної модульної системи.

У другому розділі було спроектовано архітектуру програмного продукту. Визначено основні рівні системи: web- та CLI-інтерфейси, бізнес-логіка, сховище даних, LLM-рівень і конфігураційний рівень. Також сформовано функціональні й нефункціональні вимоги, описано структуру бази даних і наведено UML-діаграми системи.

У третьому розділі було описано реалізацію програмної системи. Основну бізнес-логіку зосереджено у **ChatManager**, збереження даних реалізовано через **PostgresStorage**, а LLM-рівень — через **TransformersLLMService** та **LLMRuntime**. У web-інтерфейсі реалізовано streaming generation, зупинку генерації, пошук повідомлень, експорт діалогів, відображення Markdown, блоків коду та LaTeX-формул через локальний KaTeX.

Особливу увагу приділено розширюваності системи. Реалізовано підтримку runtime-перемикання моделей, generation presets, локального registry моделей та PEFT LoRA/QLoRA-адаптерів. Завдяки цьому система не прив'язана до однієї конкретної моделі та може бути використана для подальших експериментів з різними локальними LLM.

Працездатність системи перевірено автоматизованими тестами. Тестами покрито доменну логіку, web endpoints, storage-рівень, runtime-керування моделлю, побудову prompt-ів, конфігурацію, міграції та допоміжні скрипти. Повний набір тестів успішно виконується і підтверджує коректність основних сценаріїв роботи.

У результаті поставлену мету досягнуто. Розроблена система є працездатним навчальним програмним продуктом, який демонструє повний цикл побудови локальної LLM-системи: від проектування архітектури й бази даних до реалізації web-інтерфейсу, інтеграції локальної моделі, streaming-генерації та автоматизованого тестування.

Подальший розвиток системи може включати покращення якості відповідей шляхом підбору сильніших локальних моделей, додавання повноцінної довготривалої пам'яті між чатами, розширення механізму адаптерів, покращення UI для керування сховищем та впровадження авторизації у разі переходу від локального навчального використання до багатокористувацького сценарію.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Vaswani A., Shazeer N., Parmar N., Uszkoreit J., Jones L., Gomez A. N., Kaiser L., Polosukhin I. Attention Is All You Need. Advances in Neural Information Processing Systems, 2017. URL: <https://arxiv.org/abs/1706.03762>.
2. Zhao W. X., Zhou K., Li J., Tang T., Wang X., Hou Y. та ін. A Survey of Large Language Models. arXiv preprint, 2023. URL: <https://arxiv.org/abs/2303.18223>.
3. Wolf T., Debut L., Sanh V., Chaumond J., Delangue C., Moi A. та ін. Transformers: State-of-the-Art Natural Language Processing. Proceedings of EMNLP: System Demonstrations, 2020. URL: <https://aclanthology.org/2020.emnlp-demos.6/>.
4. Hugging Face. Transformers documentation. URL: <https://huggingface.co/docs/transformers/index>.
5. Hugging Face. Chat templates documentation. URL: https://huggingface.co/docs/transformers/en/chat_templating.
6. Hugging Face. Text generation documentation. URL: https://huggingface.co/docs/transformers/en/llm_tutorial.
7. Qwen Team. Qwen2.5 Technical Report. arXiv preprint, 2024. URL: <https://arxiv.org/abs/2412.15115>.
8. Hugging Face. Qwen2.5-0.5B-Instruct model card. URL: <https://huggingface.co/Qwen/Qwen2.5-0.5B-Instruct>.
9. Flask. Flask Documentation. URL: <https://flask.palletsprojects.com/>.
10. Flask. Quickstart. URL: <https://flask.palletsprojects.com/en/stable/quickstart/>.
11. PostgreSQL. PostgreSQL Documentation. URL: <https://www.postgresql.org/docs/>.
12. PostgreSQL. Tutorial. URL: <https://www.postgresql.org/docs/current/tutorial.html>.
13. PostgreSQL. Database Concepts. URL: <https://www.postgresql.org/docs/current/tutorial-concepts.html>.
14. Python Software Foundation. Python Documentation. URL: <https://docs.python.org/3/>.
15. PlantUML. PlantUML Documentation. URL: <https://plantuml.com/>.
16. LM Studio. Documentation. URL: <https://lmstudio.ai/docs>.
17. Ollama. Documentation. URL: <https://github.com/ollama/ollama/tree/main/docs>.
18. Open WebUI. Documentation. URL: <https://docs.openwebui.com/>.
19. SQLAlchemy. SQLAlchemy Documentation. URL: <https://docs.sqlalchemy.org/>.
20. Alembic. Alembic Documentation. URL: <https://alembic.sqlalchemy.org/>.

21. Pytest. pytest Documentation. URL: <https://docs.pytest.org/>.
22. KaTeX. KaTeX Documentation. URL: <https://katex.org/docs/>.
23. Hugging Face. PEFT documentation. URL: <https://huggingface.co/docs/peft/index>.

ДОДАТКИ

Додаток А. Структура репозиторію програмного продукту

Вихідний код програмного продукту організовано за модульним принципом. Основні каталоги репозиторію:

- `src/core` — доменні моделі та бізнес-логіка чатів;
- `src/web` — Flask routes, HTML-шаблони, CSS і JavaScript web-інтерфейсу;
- `src/cli` — консольний режим роботи;
- `src/storage` — абстракція сховища та PostgreSQL-реалізація;
- `src/llm` — LLM-сервіси, runtime-керування моделлю та model registry;
- `migrations` — Alembic-міграції бази даних;
- `tests` — автоматизовані тести;
- `scripts` — допоміжні скрипти для завантаження моделей та адаптерів.

Така структура відповідає обраному багаторівневому підходу і дозволяє окремо розвивати інтерфейс користувача, бізнес-логіку, сховище даних та LLM-рівень.

Додаток Б. Фрагменти модульних тестів

Нижче наведено приклади тестів, які перевіряють основні компоненти системи.

Тест створення чату у ChatManager:

```
def test_create_chat_with_custom_title(tmp_path):
    manager = make_manager(tmp_path)

    chat = manager.create_chat("Course Project")

    assert chat.title == "Course Project"
    assert manager.get_chat(chat.id).title == "Course Project"
```

Фрагмент перевіряє, що доменний сервіс створює чат із заданою назвою та дозволяє отримати його зі сховища.

Тест валідації дубльованих назв чатів:

```
def test_reject_duplicate_title_case_insensitively(tmp_path):
    manager = make_manager(tmp_path)
    manager.create_chat("Project Chat")

    with pytest.raises(DuplicateChatTitleError):
        manager.create_chat("project chat")
```

Фрагмент перевіряє, що система не дозволяє створювати дублікати назв чатів незалежно від регістру символів.

Тест перемикавання LoRA/QLoRA-адаптера у runtime:

```
def test_runtime_switch_applies_adapter_path(monkeypatch):
    captured = {}
    manager = make_manager()
    runtime = LLMRuntime(manager)

    def fake_create(backend, model_name_or_path=None,
                    generation_preset=None, adapter_path=None):
        captured["adapter_path"] = adapter_path
        return FakeLLMService()

    monkeypatch.setattr("src.llm.runtime.create_llm_service", fake_create)

    runtime.switch(
        "transformers",
        model_name_or_path="models/qwen",
        adapter_path="adapters/qwen-lora",
    )

    assert captured["adapter_path"] == "adapters/qwen-lora"
```

Фрагмент перевіряє, що шлях до adapter передається у фабрику LLM-сервісу під час runtime-перемикавання моделі.

Тест web endpoint для надсилання повідомлення:

```
def test_send_endpoint_adds_user_and_assistant_messages(tmp_path):
    client, manager = make_client(tmp_path)
    chat = manager.create_chat("Web chat")

    response = client.post(f"/chat/{chat.id}/send",
                           json={"message": "Hello"})

    payload = response.get_json()
    assert response.status_code == 200
    assert payload["ok"] is True
    assert payload["user_message"]["content"] == "Hello"
    assert payload["assistant_message"]["role"] == "assistant"
    assert len(manager.get_chat(chat.id).messages) == 2
```

Фрагмент перевіряє, що HTTP endpoint додає повідомлення користувача, генерує відповідь асистента і зберігає обидва повідомлення в історії чату.

Тест сканування локальної моделі:

```
def test_folder_with_config_and_safetensors_is_listed(tmp_path):
    model_dir = tmp_path / "models" / "qwen"
    model_dir.mkdir(parents=True)
    (model_dir / "config.json").write_text("{} ", encoding="utf-8")
    (model_dir / "model.safetensors").write_text("fake", encoding="utf-8")
    (model_dir / "tokenizer.json").write_text("{} ", encoding="utf-8")

    models = list_local_models(tmp_path / "models")

    assert len(models) == 1
    assert models[0].name == "qwen"
    assert models[0].has_config is True
    assert models[0].has_weights is True
    assert models[0].has_tokenizer is True
```

Фрагмент перевіряє, що model registry знаходить локальну модель лише тоді, коли в каталозі є конфігурація, ваги та tokenizer.

Додаток В. Результати покриття коду тестами

Для оцінювання покриття було використано coverage.py. Повний запуск тестів:

```
python -m coverage run -m pytest
```

Результат виконання:

```
105 passed in 3.26s
```

Звіт покриття для production-коду було сформовано командою:

```
python -m coverage report --include="src/*,migrations/*,scripts/*"
```

Основні результати:

Фрагмент звіту покриття production-коду

Модуль	Покриття
src/core/chat_manager.py	94%
src/config.py	98%
src/llm/runtime.py	97%
src/llm/model_registry.py	92%
src/storage/postgres_storage.py	77%
src/web/web_app.py	81%
Загальне покриття production-коду	77%

Отримане покриття відповідає рекомендованому рівню 70–80% і підтверджує, що основна бізнес-логіка та ключові інтеграційні точки системи перевіряються автоматизованими тестами.

Додаток Г. Приклади функціонального тестування web/API

Функціональні сценарії перевірялися через Flask test client. Вони імітують HTTP-запити користувача до web-застосунку.

Приклади функціонального тестування

Дія	Метод	Endpoint	Очікуваний результат
Відкрити головну сторінку	GET	/	Статус 200, відображення sidebar, model status і списку чатів.
Створити чат	POST	/chats	Новий чат створено, користувача перенаправлено на сторінку чату.
Надіслати повідомлення	POST	/chat/<id>/send	Збережено повідомлення користувача й відповідь асистента.
Streaming-генерація	POST	/chat/<id>/stream	Відповідь повертається NDJSON-частинами та зберігається після завершення.
Зупинити генерацію	POST	/api/generation/stop	Активний stop event встановлено, генерація завершується контрольовано.
Перемкнути модель	POST	/api/model/switch	Runtime переходить у стан завантаження, після успіху активна модель оновлюється.
Пошук повідомлень	GET	/api/messages/search	Повертаються релевантні повідомлення з назвою чату і роллю автора.

Експорт чату	GET	/chat/<id>/export	Користувач отримує Markdown-файл з історією діалогу.
--------------	-----	-------------------	--

Приклад перевірки помилки валідації порожнього повідомлення:

```
def test_send_endpoint_rejects_empty_message(tmp_path):
    client, manager = make_client(tmp_path)
    chat = manager.create_chat("Web chat")

    response = client.post(f"/chat/{chat.id}/send",
                           json={"message": "  "})

    assert response.status_code == 400
    assert response.get_json() == {
        "ok": False,
        "error": "Message cannot be empty.",
    }
```

Цей тест перевіряє, що web-рівень не приймає порожні повідомлення і повертає користувачу коректну помилку.